Balanced Brackets

AlgoExpert · Medium · Stacks

The problem

- Are (), [], {} properly balanced and nested?
- Non-bracket chars are ignored.

Intuition

- A stack matches each closing bracket with the most recent opening one.

Brute force vs optimal

- Repeatedly delete '()', '[]', '{}' → $O(n^2)$
- Stack of openers → O(n)
- Only brackets matter; ignore other chars

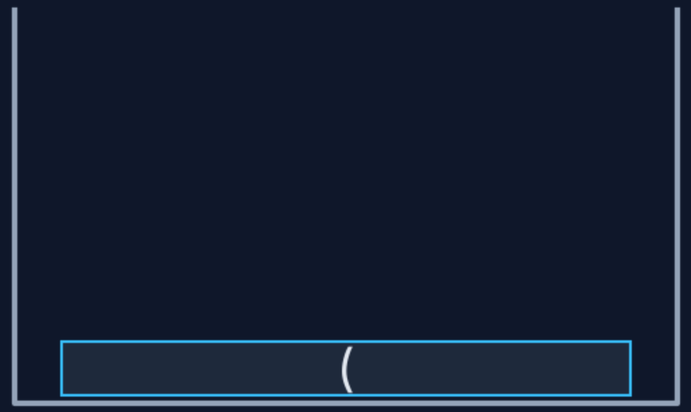
Approach

- 1. Push openers
- 2. On a closer, pop and compare
- 3. Ignore other characters
- 4. Valid if the stack ends empty

Worked example



stack (top ↑)

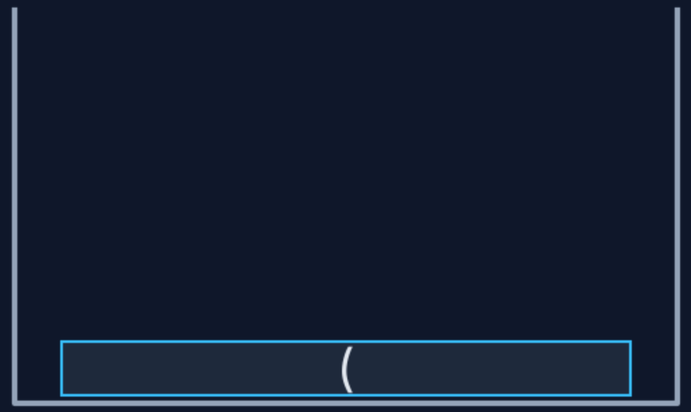


'(': push

Worked example



stack (top ↑)

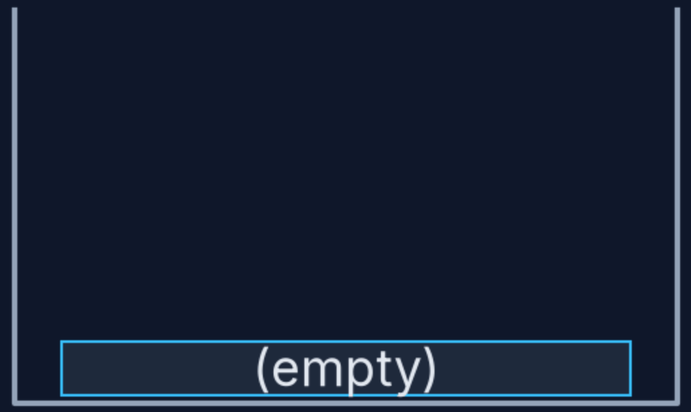


']': pop & match

Worked example



stack (top ↑)

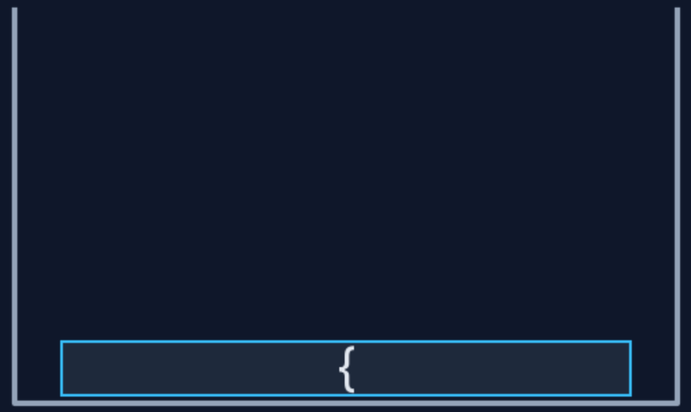


')': pop & match

Worked example



stack (top ↑)

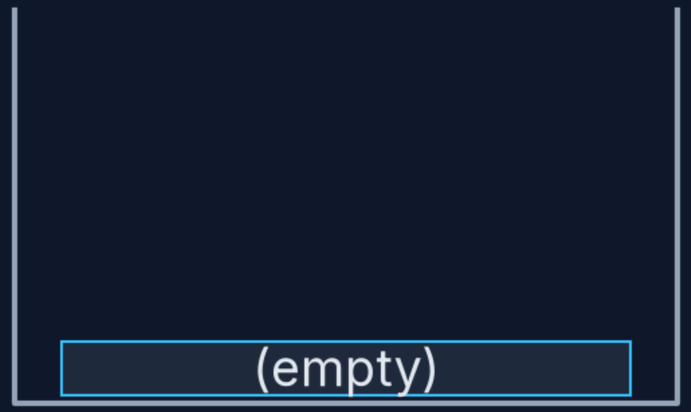


'{': push

Worked example



stack (top ↑)



')': pop & match

Java solution

```java
public class BalancedBrackets {
    public static boolean balancedBrackets(String s) {
        ArrayDeque<Character> st = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if ("([{".indexOf(c) >= 0) st.push(c);
            else if (")]}".indexOf(c) >= 0) {
                if (st.isEmpty() || "([{".indexOf(st.pop()) != ")]}".indexOf(c)) return false;
            }
        }
        return st.isEmpty();
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- Closer on empty stack → false
- Leftover openers → false
- Non-bracket characters ignored

Interview tips & follow-ups

- Follow-up: minimum insertions to balance

Recap

- A stack matches each closing bracket with the most recent opening one.
- Complexity: O(n)